



ENTERPRISE DATABASE ARCHITECTURE & GOVERNANCE

SQL Server Instance Design · Backup · Replication · Security · Monitoring

Sevinj

Sections 1–2

Vafa

Sections 3–5

Umman

Sections 6–8

PROJECT AGENDA

00 introduction

01 System Configuration & Schema Design *Sevinj*

02 Backup Strategy & Workflow
Sevinj

03 Replication & Analytical Data Flow
Vafa

04 Automated Maintenance Plan
Vafa

05 Database Mail & Alerting Infrastructure
Vafa

06 Resource Governance & Session Tracking
Umman

07 Advanced Monitoring & Optimization
Umman

08 Security, Auditing & Encryption
Umman

System Configuration & Backup Strategy

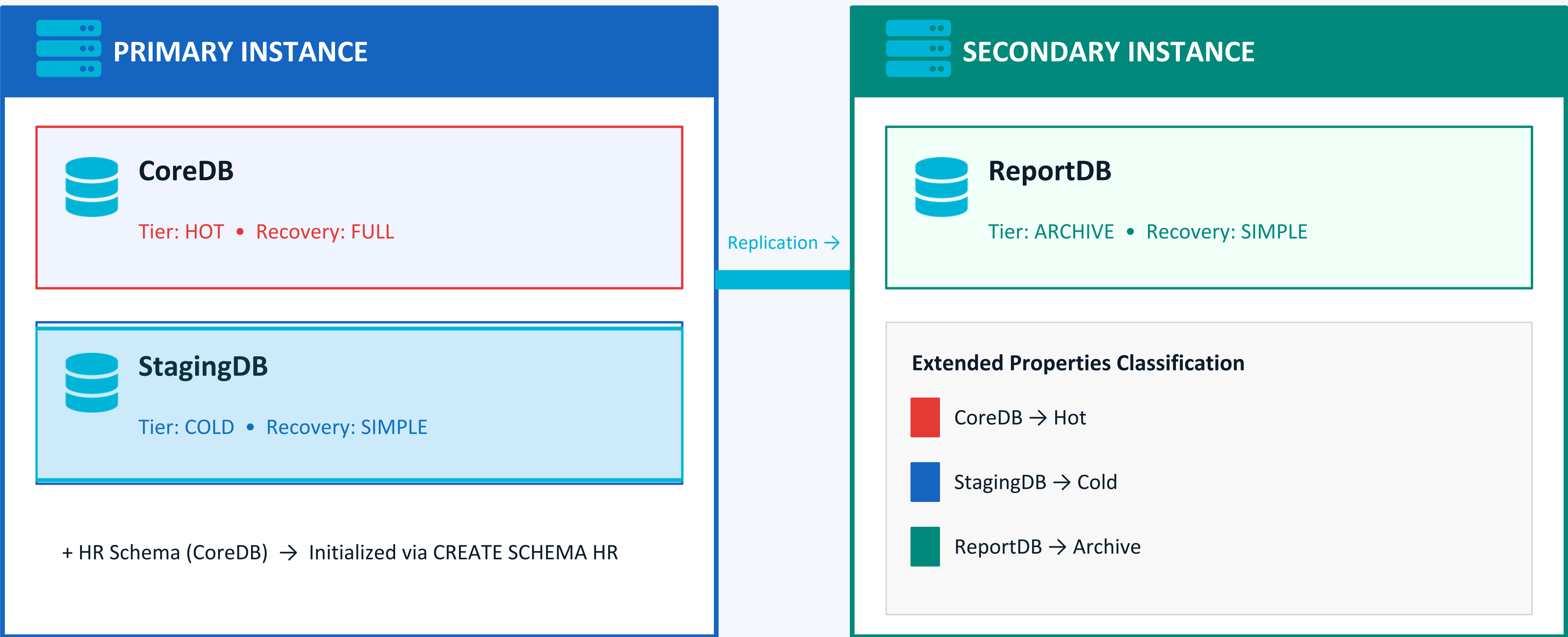
Topics covered:

- Dual SQL Server Instance Architecture
- Database Distribution (Primary / Secondary)
- HR Schema Initialization
- Extended Properties (Hot / Cold / Archive)
- Recovery Model Mapping
- Backup Tiers (Hot, Cold, Archive)
- Visual Backup Workflow Diagram



Instance Architecture

Dual SQL Server Node Design



System Configuration & Backup Strategy

01.setup_primary.sql

```
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'CoreDB')
BEGIN
    CREATE DATABASE CoreDB;
END
GO

ALTER DATABASE CoreDB SET RECOVERY FULL;
GO

IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'StagingDB')
BEGIN
    CREATE DATABASE StagingDB;
END
GO

ALTER DATABASE StagingDB SET RECOVERY SIMPLE;
GO
```

Presenter: Sevinj

```
USE CoreDB;
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'HR')
BEGIN
    EXEC('CREATE SCHEMA HR');
END
GO

IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[HR].[Employees]') AND type in (N'U'))
BEGIN
    CREATE TABLE HR.Employees (
        EmployeeID INT PRIMARY KEY IDENTITY(1,1),
        FirstName NVARCHAR(50),
        LastName NVARCHAR(50),
        Email NVARCHAR(100),
        Salary DECIMAL(18, 2),
        HireDate DATE DEFAULT GETDATE()
    );
END
GO

EXEC sys.sp_addextendedproperty
    @name = N'Tier',
    @value = N'Hot';
GO

USE StagingDB;
GO

EXEC sys.sp_addextendedproperty
    @name = N'Tier',
    @value = N'Cold';
GO
```


System Configuration & Backup Strategy

01.setup_primary.sql

```
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'CoreDB')
BEGIN
    CREATE DATABASE CoreDB;
END
GO

ALTER DATABASE CoreDB SET RECOVERY FULL;
GO

IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'StagingDB')
BEGIN
    CREATE DATABASE StagingDB;
END
GO

ALTER DATABASE StagingDB SET RECOVERY SIMPLE;
GO
```

Presenter: Sevinj

```
USE CoreDB;
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'HR')
BEGIN
    EXEC('CREATE SCHEMA HR');
END
GO

IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[HR].[Employees]') AND type in (N'U'))
BEGIN
    CREATE TABLE HR.Employees (
        EmployeeID INT PRIMARY KEY IDENTITY(1,1),
        FirstName NVARCHAR(50),
        LastName NVARCHAR(50),
        Email NVARCHAR(100),
        Salary DECIMAL(18, 2),
        HireDate DATE DEFAULT GETDATE()
    );
END
GO

EXEC sys.sp_addextendedproperty
    @name = N'Tier',
    @value = N'Hot';
GO

USE StagingDB;
GO

EXEC sys.sp_addextendedproperty
    @name = N'Tier',
    @value = N'Cold';
GO
```

Schema Design & Metadata Classification

01.setup_secondary.sql

Blame

15 lines (13 loc) · 261 Bytes



```
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'ReportDB')
BEGIN
    CREATE DATABASE ReportDB;
END
GO

ALTER DATABASE ReportDB SET RECOVERY SIMPLE;
GO

USE ReportDB;
GO
EXEC sys.sp_addextendedproperty
    @name = N'Tier',
    @value = N'Archive';
GO
```

Presenter: Sevinj

HR Schema

- Namespace isolation within CoreDB
- Contains: Employees, Departments, Salaries
- Owned by dbo — access via HR.Employees

Recovery Models

- Full Recovery → CoreDB (Hot)
- Simple Recovery → StagingDB (Cold)
- Simple Recovery → ReportDB (Archive)
- Drives backup complexity per tier

Extended Properties

- sp_addextendedproperty to tag each DB
- Hot → full point-in-time recovery
- Cold/Archive → simplified retention

SECTION 1 & 2

Backup Strategy

02.backup_strategy_primary.sql

Blame 34 lines (28 loc) · 2.69 KB



```
USE msdb;
GO

EXEC dbo.sp_add_job @job_name = N'CORE_Full_Backup', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'CORE_Full_Backup', @step_name = N'BackupStep',
    @command = N'BACKUP DATABASE [CoreDB] TO DISK = N''/var/opt/mssql/data/CoreDB_Full.bak'' WITH INIT, COMPRESSION, STATS = 10';
EXEC dbo.sp_add_jobschedule @job_name = N'CORE_Full_Backup', @name = N'Sched_Daily_00', @freq_type = 4, @freq_interval = 1, @active_start_time = 000000;

EXEC dbo.sp_add_job @job_name = N'CORE_Diff_Backup', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'CORE_Diff_Backup', @step_name = N'BackupStep',
    @command = N'BACKUP DATABASE [CoreDB] TO DISK = N''/var/opt/mssql/data/CoreDB_Diff.bak'' WITH DIFFERENTIAL, INIT, COMPRESSION, STATS = 10';
EXEC dbo.sp_add_jobschedule @job_name = N'CORE_Diff_Backup', @name = N'Sched_6HR', @freq_type = 4, @freq_interval = 1, @active_start_time = 060000;

EXEC dbo.sp_add_job @job_name = N'CORE_Log_Backup', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'CORE_Log_Backup', @step_name = N'BackupStep',
    @command = N'BACKUP LOG [CoreDB] TO DISK = N''/var/opt/mssql/data/CoreDB_Log.trn'' WITH NOINIT, COMPRESSION, STATS = 10';
EXEC dbo.sp_add_jobschedule @job_name = N'CORE_Log_Backup', @name = N'Sched_15MIN', @freq_type = 4, @freq_interval = 1, @freq_subday_type = 4, @freq_subday_interval = 15;

EXEC dbo.sp_add_job @job_name = N'STAGING_Weekly_Full', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'STAGING_Weekly_Full', @step_name = N'BackupStep',
    @command = N'BACKUP DATABASE [StagingDB] TO DISK = N''/var/opt/mssql/data/StagingDB_Full.bak'' WITH INIT, COMPRESSION, STATS = 10';
EXEC dbo.sp_add_jobschedule @job_name = N'STAGING_Weekly_Full', @name = N'Sched_SUN_01', @freq_type = 8, @freq_interval = 1, @freq_recurrence_factor = 1, @active_start_time = 010000;

EXEC dbo.sp_add_job @job_name = N'STAGING_Weekly_Diff', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'STAGING_Weekly_Diff', @step_name = N'BackupStep',
    @command = N'BACKUP DATABASE [StagingDB] TO DISK = N''/var/opt/mssql/data/StagingDB_Diff.bak'' WITH DIFFERENTIAL, INIT, COMPRESSION, STATS = 10';
EXEC dbo.sp_add_jobschedule @job_name = N'STAGING_Weekly_Diff', @name = N'Sched_WED_01', @freq_type = 8, @freq_interval = 8, @freq_recurrence_factor = 1, @active_start_time = 010000;
```


Backup Strategy

02.backup_strategy_secondary.sql

```
Code Blame 12 lines (9 loc) · 608 Bytes 
1  USE msdb;
2  GO
3
4  EXEC dbo.sp_add_job @job_name = N'REPORT_Monthly_Archive', @enabled = 1;
5  EXEC dbo.sp_add_jobstep @job_name = N'REPORT_Monthly_Archive', @step_name = N'BackupStep',
6      @command = N'BACKUP DATABASE [ReportDB] TO DISK = N''/var/opt/mssql/data/ReportDB_Archive.bak'' WITH INIT, COMPRESSION, STATS = 10';
7
8  EXEC dbo.sp_add_jobschedule @job_name = N'REPORT_Monthly_Archive', @name = N'Sched_Monthly_01',
9      @freq_type = 16, @freq_interval = 1, @freq_recurrence_factor = 1, @active_start_time = 020000;
10
11 EXEC dbo.sp_add_jobserver @job_name = N'REPORT_Monthly_Archive', @server_name = N'(LOCAL)';
12 GO
```



Backup Strategy & Workflow



HOT TIER

CoreDB

Schedule: Daily

1. Full Backup



2. Differential Backup



3. Transaction Log Backup



COLD TIER

StagingDB

Schedule: Weekly

1. Full Backup



2. Differential Backup



ARCHIVE TIER

ReportDB

Schedule: Monthly

1. Full Backup Only



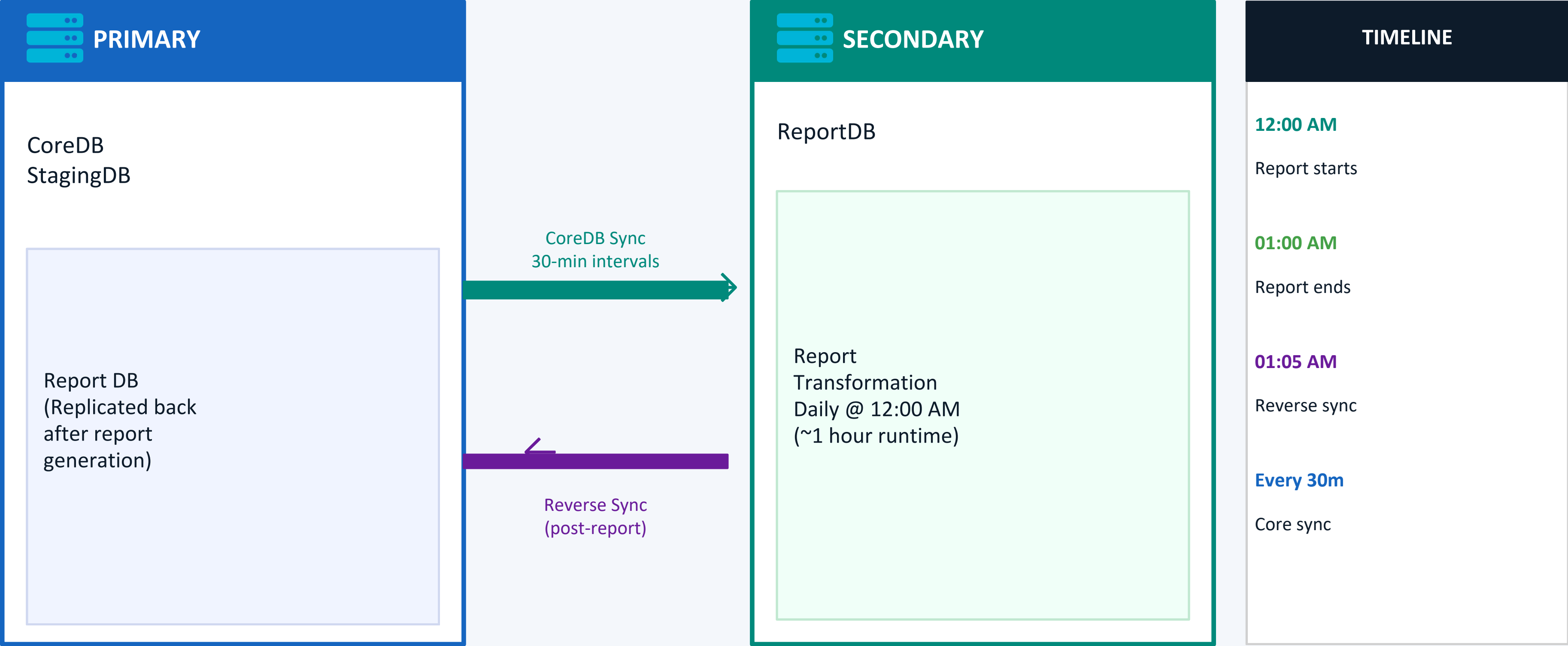
Replication · Maintenance · Mail & Alerting

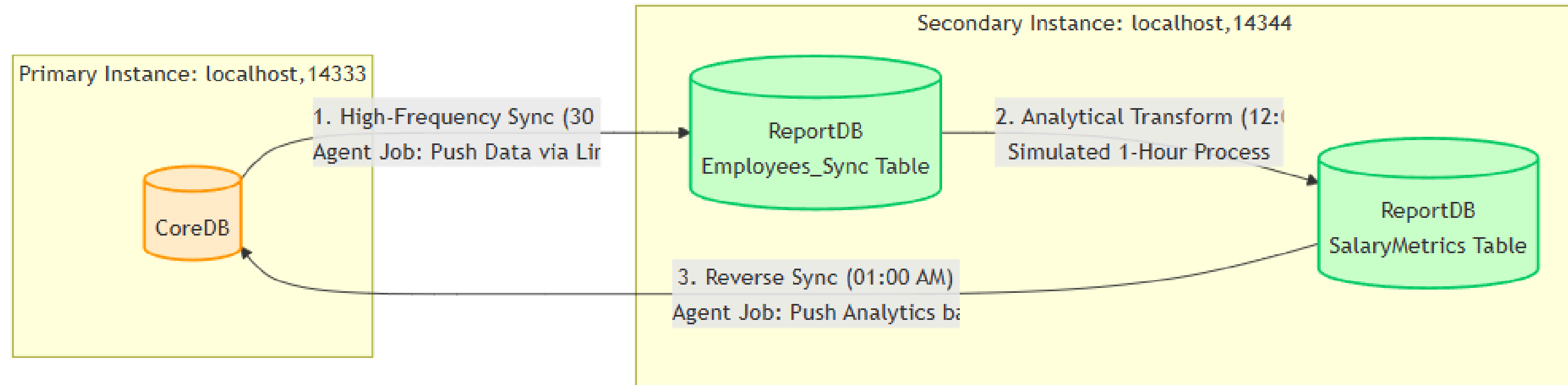
Topics covered:

- Near-Real-Time Replication (30-min intervals)
- Core DB → Secondary Synchronization
- Daily Report Transformation (12:00 AM)
- Reverse Sync: Report DB → Primary
- Reindex, Update Stats, DBCC CHECKDB
- History Cleanup — Weekend Midnight
- Database Mail SMTP Configuration
- Admin Operator with 3-Email Distribution



Replication & Analytical Data Flow





- **Performance Isolation:** Analytical queries like AVG or SUM on millions of rows are "expensive" and can lock tables. By moving this workload, we ensure zero impact on the end-users of the application.
- **Data Freshness:** The 30-minute synchronization prevents the reporting data from becoming "stale". It provides decision-makers with near-real-time accuracy rather than waiting for a 24-hour update cycle.
- **Centralized Insights:** The Reverse Sync at the end of the cycle creates an automated feedback loop. This delivers the final report results directly back to the Primary database so the business can access insights without switching servers.

Replication Implementation on Primary Node

Script: 03_replication.sql

```
EXEC master.dbo.sp_addlinkedserver
    @server=N'SECONDARY_NODE',
    @datasrc=N'dba_secondary';

EXEC master.dbo.sp_addlinkedsrvlogin
    @rmtsrvname=N'SECONDARY_NODE',
    @useself=N'False',
    @rmtuser=N'sa',

    @rmtpassword=N'StrongP@ssw0rd!2026';
```

```
EXEC dbo.sp_add_job @job_name =
N'REPLICATION_30MIN_SYNC', @enabled =
1;

EXEC dbo.sp_add_jobschedule @job_name
= N'REPLICATION_30MIN_SYNC',
    @freq_subday_type = 4,
    @freq_subday_interval = 30; --
30-minute repeat
```

1. The Connection Bridge

To move data, the Primary must first establish a secure "handshake" with the Secondary Node.

3. Automating the 30-Minute Cycle

Setting the interval to 30 minutes ensures the reporting database is never "stale," fulfilling the requirement for near-real-time synchronization.

Replication Implementation on Primary Node

Script: 03_replication.sql

```
MERGE SECONDARY_NODE.ReportDB.dbo.Employees_Sync AS tgt
USING (
    SELECT EmployeeID, FirstName, LastName, Email, Salary
    FROM CoreDB.HR.Employees
) AS src
ON tgt.EmployeeID = src.EmployeeID
WHEN MATCHED AND (
    ISNULL(tgt.FirstName, N'') <> ISNULL(src.FirstName, N'')
    OR ISNULL(tgt.LastName, N'') <> ISNULL(src.LastName, N'')
    OR ISNULL(tgt.Email, N'') <> ISNULL(src.Email, N'')
    OR ISNULL(tgt.Salary, 0) <> ISNULL(src.Salary, 0)
)
    THEN UPDATE SET
        tgt.FirstName = src.FirstName,
        tgt.LastName = src.LastName,
        tgt.Email = src.Email,
        tgt.Salary = src.Salary
WHEN NOT MATCHED BY TARGET
    THEN INSERT (EmployeeID, FirstName, LastName, Email, Salary)
        VALUES (src.EmployeeID, src.FirstName, src.LastName,
src.Email, src.Salary)
WHEN NOT MATCHED BY SOURCE
    THEN DELETE;
',
```

2. The Smart Sync (The MERGE Statement)

- Matched: Updates records if the salary or names changed.
- New: Inserts rows that don't exist on the Secondary yet.
- Deleted: Removes rows from the Secondary that were deleted from the Primary.

Replication Implementation on Secondary Node

Script: 03_replication.sql

1. Establishing the Return Path

```
EXEC master.dbo.sp_addlinkedserver
    @server=N'PRIMARY_NODE',
    @datasrc=N'dba_primary';

EXEC master.dbo.sp_addlinkedsrvlogin
    @rmtsrvname=N'PRIMARY_NODE',
    @useself=N'False',
    @rmtuser=N'sa',
    @rmtpassword=N'StrongP@ssw0rd!2026';
```

2. The Analytical Workspace

```
CREATE TABLE dbo.Employees_Sync (
    EmployeeID INT PRIMARY KEY,
    FirstName NVARCHAR(50),
    Salary DECIMAL(18, 2)
);
```

4.Reverse Sync and Scheduling at Midnight

3.Transform (Processing)

```
WAITFOR DELAY '01:00:00'; --
Simulates heavy work
SELECT AVG(Salary) as AvgSalary
INTO ReportDB.dbo.SalaryMetrics
FROM ReportDB.dbo.Employees_Sync;
```

```
SELECT * INTO
PRIMARY_NODE.CoreDB.dbo.FinalizedReport
FROM ReportDB.dbo.SalaryMetrics;
EXEC dbo.sp_add_jobschedule @job_name =
N'MIDNIGHT_DATA_FLOW',
    @name = N'Sched_Daily_00',
    @active_start_time = 000000; -- 12:00 AM
```


Automated Maintenance Plan

Script: 04_maintenance_plan.sql | Every Weekend @ Midnight



Integrity Check

- DBCC CHECKDB on all databases
- Detects allocation corruption
- Reports page-level errors
- WITH NO_INFOMSGS for clean logs

Update Statistics

- sp_updatestats system proc
- Refreshes cardinality estimates
- Improves optimizer plan choices
- Runs after Reindex

Reindex

- ALTER INDEX ALL ON ? REBUILD
- Rebuilds all indexes per table
- Eliminates fragmentation
- Improves query seek performance



History Cleanup

- sp_purge_jobhistory call
- Removes records > 30 days old
- Prevents msdb bloat
- Keeps Agent history manageable

Execution Order: Backup completes first → DBCC CHECKDB→Reindex → Update Stats→ History Cleanup | Sequential to prevent lock contention

Maintenance Script

Script: 04_maintenance_plan.sql

1. Full Backup

```
EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name =
N'POST_BACKUP_CORE_FULL',
    @command = N'BACKUP DATABASE [CoreDB] TO DISK =
N''/var/opt/mssql/data/CoreDB_WeekendFull.bak'' WITH INIT, COMPRESSION, STATS = 10;',
    @on_success_action = 3;

EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name =
N'POST_BACKUP_STAGING_FULL',
    @command = N'BACKUP DATABASE [StagingDB] TO DISK =
N''/var/opt/mssql/data/StagingDB_WeekendFull.bak'' WITH INIT, COMPRESSION, STATS =
10;',
    @on_success_action = 3;
```

2.Integrity Check (DBCC CHECKDB)

```
EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name =
N'DBCC_CHECK',
    @command = N'DBCC CHECKDB (CoreDB) WITH NO_INFOMSGS; DBCC CHECKDB (StagingDB) WITH
NO_INFOMSGS;',
    @on_success_action = 3;
```

sp_MSforeachtable

- Loops over every user table
- Executes REBUILD on all indexes
- Single command for all tables
- Compacts index pages

sp_updatestats

- Native system procedure
- Updates row count estimates
- Forces optimizer recompile
- No table locks required

DBCC CHECKDB

- Validates allocation structures
- Checks catalog consistency
- Reports corruption to error log
- WITH NO_INFOMSGS = quiet mode

Weekend Window

- Saturday/Sunday 00:00 AM | No user contention

Maintenance Script

Script: 04_maintenance_plan.sql

3. Index Reorganization (sp_MSforeachtable)

```
EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name =  
N'INDEX_REORG',  
    @command = N'USE CoreDB; EXEC sp_MSforeachtable ''ALTER INDEX ALL ON ?  
REORGANIZE'';',  
    @on_success_action = 3;
```

4. Update Statistics (sp_updatestats)

```
EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name =  
N'DBCC_CHECK',  
    @command = N'DBCC CHECKDB (CoreDB) WITH NO_INFOMSGS; DBCC CHECKDB (StagingDB) WITH  
NO_INFOMSGS;',  
    @on_success_action = 3;
```

sp_MSforeachtable

- Loops over every user table
- Executes REBUILD on all indexes
- Single command for all tables
- Compacts index pages

sp_updatestats

- Native system procedure
- Updates row count estimates
- Forces optimizer recompile
- No table locks required

DBCC CHECKDB

- Validates allocation structures
- Checks catalog consistency
- Reports corruption to error log
- WITH NO_INFOMSGS = quiet mode

Weekend Window

- Saturday/Sunday 00:00 AM | No user contention

Maintenance Script

Script: 04_maintenance_plan.sql

5. System Cleanup (msdb Purging)

```
EXEC dbo.sp_add_jobstep @job_name = N'WEEKLY_DB_MAINTENANCE', @step_name = N'CLEANUP',
    @command = N'EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = ''2026-01-01'';
EXEC msdb.dbo.sp_purge_jobhistory @oldest_date = ''2026-01-01'';;
```

6. Scheduling & Server Assignment

```
EXEC dbo.sp_add_jobschedule @job_name = N'WEEKLY_DB_MAINTENANCE', @name =
N'Sched_SAT_00',
    @freq_type = 8, @freq_interval = 64, @freq_recurrence_factor = 1,
    @active_start_time = 000000;

EXEC dbo.sp_add_jobserver @job_name = N'WEEKLY_DB_MAINTENANCE', @server_name =
N'(LOCAL)';
```

sp_MSforeachtable

- Loops over every user table
- Executes REBUILD on all indexes
- Single command for all tables
- Compacts index pages

sp_updatestats

- Native system procedure
- Updates row count estimates
- Forces optimizer recompile
- No table locks required

DBCC CHECKDB

- Validates allocation structures
- Checks catalog consistency
- Reports corruption to error log
- WITH NO_INFOMSGS = quiet mode

Weekend Window

- Saturday/Sunday 00:00 AM | No user contention

Automated Maintenance Plan

Select a page	Script ▼ ? Help				
⚙ General ⚙ Steps ⚙ Schedules ⚙ Alerts ⚙ Notifications ⚙ Targets	Job step list:				
	S...	Name	Type	On Succ...	On Failure
	1	DBCC_CHECK	Transac...	Go to th...	Quit the...
	2	INDEX_REORG	Transac...	Go to th...	Quit the...
	3	STATS_UPDATE	Transac...	Go to th...	Quit the...
	4	CLEANUP	Transac...	Quit the...	Quit the...

The sequential steps of the WEEKLY_DB_MAINTENANCE SQL Agent Job.

Database Mail & Alerting Infrastructure

Script: 05_database_mail.sql

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'Database Mail XPs', 1;

EXECUTE msdb.dbo.sysmail_add_account_sp
    @account_name = 'AlertAccount',
    @email_address = 'EMAIL',
    @display_name = 'SQL Alert System',
    @mailserver_name = 'SERVER',
    @port = 587,
    @enable_ssl = 1,
    @username = 'USERNAME',
    @password = 'PASSWORD';

EXECUTE msdb.dbo.sysmail_add_profile_sp
    @profile_name = 'AdminProfile';

EXECUTE msdb.dbo.sysmail_add_profileaccount_sp
    @profile_name = 'AdminProfile',
    @account_name = 'AlertAccount',
    @sequence_number = 1;

EXEC msdb.dbo.sp_add_operator
    @name = N'DBA_Team',
    @enabled = 1,
    @email_address = N'admin1@lab.local;admin2@lab.local;admin3@lab.local';

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'AdminProfile',
    @recipients = 'admin1@lab.local',
    @body = 'System Check: Mail service initialized.',
    @subject = 'DBA Alert Test';
```

Mail Profile

- Profile: DBAAlerts
- Linked to SMTP account
- All Agent job alerts route here
- Can have multiple accounts for failover

Admin Operator

- sp_add_operator creates operator
- Three emails: ; separated
- Receives: Job success/failure alerts
- Receives: Blocked process alerts
- Receives: Deadlock notifications

SMTP Configuration

- Server: smtp.company.com
- Port: 587 | SSL: Enabled
- Authentication: Secure credentials
- End-to-end test with sp_send_dbmail

Database Mail & Alerting Infrastructure

DBA Alert Test - SUCCESS



From: SQL Alert System <dba-alerts@lab.local>

To: <admin1@lab.local>

2026-04-30 20:08, 371 Bytes

Show Headers

HTML

HTML Source

Text

Raw

Spam Analysis

Tech Info



System Check: Mail service initialized and Mailtrap connected successfully.

Governance · Monitoring · Security & Encryption

Topics covered:

- Resource Governor: Deploy User Restriction
- Block & Deadlock Real-Time Monitoring
- Extended Events (XEEvents) — Blocked Processes
- Backup Audit Job Success/Failure Tracking
- DB Ownership Self-Healing Script
- Login & Privilege Audit Report
- SQL Server Audit — HR.Employees SELECT tracking
- Column Encryption: Salary Field (Always Encrypted + TDE)



Resource Governance & Session Tracking

Script: 06_resource_governance.sql

```
/* RESOURCE GOVERNOR: CPU & Memory Limiting */
CREATE RESOURCE POOL ReportPool WITH (
    MAX_CPU_PERCENT = 20,
    CAP_CPU_PERCENT = 20,
    MAX_MEMORY_PERCENT = 20
);
GO

/* CLASSIFIER FUNCTION: Time-based workload routing */
CREATE FUNCTION dbo.RG_Classifier()
RETURNS SYSNAME WITH SCHEMABINDING AS
BEGIN
    -- Redirect deploy_user to ReportPool during midnight hour
    IF SUSER_SNAME() = N'deploy_user'
        AND CAST(GETDATE() AS TIME) >= '00:00:00'
        AND CAST(GETDATE() AS TIME) < '01:00:00'
        RETURN N'DeployGroup';
    RETURN N'default';
END;
GO

ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = dbo.RG_Classifier);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO

/* REAL-TIME DEADLOCK CAPTURE */
CREATE EVENT SESSION CaptureDeadlocks ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.event_file(SET filename = N'/var/opt/mssql/data/Deadlocks.xel')
WITH (STARTUP_STATE = ON);
GO
```

Workload Limiting

- Resource Governor pool: ReportPool
- MAX_CPU_PERCENT = 20% for deploy user
- Prevents report queries starving OLTP
- Active only during 12:00–01:00 AM window

Block Detection

- blocked process threshold = 5 seconds
- SQL Server raises event automatically
- sp_add_alert traps Performance Condition (Blocked > 0)
- Immediately emails Admin Operator

Deadlock Tracing

- Deadlock graph captured via XEvents (xml_deadlock_report)
- XEvents also trap lock_deadlock events
- Automated email on deadlock detection
- Graphs stored in .xel event file

Advanced Monitoring & Optimization

Script: 07_monitoring.sql

```
/* MONITORING: Blocked Process Reports (Threshold: 5s) */
EXEC sp_configure 'blocked process threshold', 5; RECONFIGURE;
GO

CREATE EVENT SESSION CaptureBlocks ON SERVER
ADD EVENT sqlserver.blocked_process_report
ADD TARGET package0.event_file(SET filename = N'/var/opt/mssql/data/Blocks.xel');
GO

/* BACKUP AUDIT: Tracking Daily Success/Failure */
CREATE PROCEDURE dbo.SP_AUDIT_BACKUPS AS
BEGIN
    INSERT INTO dbo.BACKUP_AUDIT_LOG (JobName, Status)
    SELECT
        j.name,
        CASE h.run_status
            WHEN 0 THEN 'FAILURE'
            WHEN 1 THEN 'SUCCESS'
            ELSE 'OTHER'
        END
    FROM msdb.dbo.sysjobhistory h
    JOIN msdb.dbo.sysjobs j ON h.job_id = j.job_id
    WHERE j.name LIKE '%Backup%'
        AND h.step_id = 0
        AND msdb.dbo.agent_datetime(h.run_date, h.run_time) > DATEADD(hh, -24,
GETDATE());
END;
GO
```

Extended Events Session

- Captures blocked_process_report event
- Filter: duration > 5000ms (5 seconds)
- Event File (.xel) target: 50 MB in-memory
- Startup state ON — always active

Backup Audit Query

- Queries msdb.dbo.sysjobhistory
- Filters on step_id = 0 (job completion)
- run_status = 1 = Success
- Any other = FAILED → alert fires

Analysis Workflow

- XEvent data read from ring_buffer
- XML parsed for blocking SQL text
- Root-cause query identified & indexed
- Performance ticket auto-generated

Security, Auditing & Encryption

Script: 08_security.sql

```
/* SELF-HEALING: Enforce Database Ownership (sa) */
CREATE PROCEDURE dbo.SP_ENFORCE_DB_OWNERSHIP AS
BEGIN
    DECLARE @db SYSNAME, @owner SYSNAME;
    DECLARE c CURSOR FOR SELECT name, SUSER_SNAME(owner_sid) FROM sys.databases WHERE
database_id > 4;
    OPEN c; FETCH NEXT FROM c INTO @db, @owner;
    WHILE @@FETCH_STATUS = 0 BEGIN
        IF @owner != 'sa' EXEC('ALTER AUTHORIZATION ON DATABASE::[' + @db + '] TO
[sa]');
        FETCH NEXT FROM c INTO @db, @owner;
    END
    CLOSE c; DEALLOCATE c;
END;
GO

/* COLUMN-LEVEL ENCRYPTION: AES-256 */
CREATE SYMMETRIC KEY SalaryKey WITH ALGORITHM = AES_256
ENCRYPTION BY CERTIFICATE SalaryCert;
GO

OPEN SYMMETRIC KEY SalaryKey DECRYPTION BY CERTIFICATE SalaryCert;
UPDATE HR.Employees SET Salary_Enc = ENCRYPTBYKEY(KEY_GUID('SalaryKey'), CAST(Salary AS
NVARCHAR(50)));
CLOSE SYMMETRIC KEY SalaryKey;
GO
```

Self-Healing Ownership

- Checks SUSER_SNAME(owner_sid)
- If NOT sa → ALTER AUTHORIZATION
- Corrects ownership to sa automatically
- Sends email alert post-correction

SQL Server Audit

- SERVER AUDIT writes to file target
- DATABASE AUDIT SPECIFICATION
- Tracks SELECT on HR.Employees
- Stores: login, query, timestamp

Cell-Level Encryption

- Column Master Key (CMK) in Cert Store
- Column Encryption Key (CEK) created
- Salary column encrypted at rest & in transit
- TDE also applied for disk-level security

Reporting

Script: 99_report.sql

```
/* SERVER PERMISSIONS AUDIT */
SELECT
    p.name AS Principal,
    perm.permission_name AS Permission,
    perm.state_desc AS State
FROM sys.server_principals p
JOIN sys.server_permissions perm ON p.principal_id = perm.grantee_principal_id
WHERE p.name NOT LIKE '##%';
GO

/* CROSS-DATABASE AUDIT: Dynamic SQL Execution */
DECLARE @DynamicSQL NVARCHAR(MAX) = N'';
SELECT @DynamicSQL = @DynamicSQL + N'
USE ' + QUOTENAME(name) + N';
INSERT INTO #DatabaseRoleMemberships
SELECT DB_NAME(), u.name, r.name
FROM sys.database_principals u
JOIN sys.database_role_members m ON u.principal_id = m.member_principal_id
JOIN sys.database_principals r ON m.role_principal_id = r.principal_id;'
FROM sys.databases WHERE database_id > 4 AND state_desc = 'ONLINE';

EXEC sys.sp_executesql @DynamicSQL;
GO
```

Server-Level Audit

- Maps logins to permissions
- Filters internal system accounts
- Reports GRANT/DENY states
- Provides instance-wide visibility

Dynamic Discovery

- Scans all user databases
- Automated SQL generation
- Skips offline/recovery nodes
- Bypasses system databases

Membership Mapping

- Tracks role-based access
- Identifies user inheritance
- Centralizes local security data
- Unified cross-DB reporting

Architecture Summary



Foundation

- Dual Instance Architecture
- HR Schema in CoreDB
- Extended Properties (Hot/Cold/Archive)
- Recovery Model Mapping



Availability

- Tiered Backup (Hot/Cold/Archive)
- 30-min Core Replication
- Daily Report Transformation
- Reverse Sync to Primary



Operations

- Weekend Maintenance Windows
- Sequential Task Execution
- Database Mail & Admin Operator
- Automated Agent Job Scheduling



Governance

- Resource Governor Limits
- Block/Deadlock Alerting
- XEvents Monitoring
- Backup Success Auditing



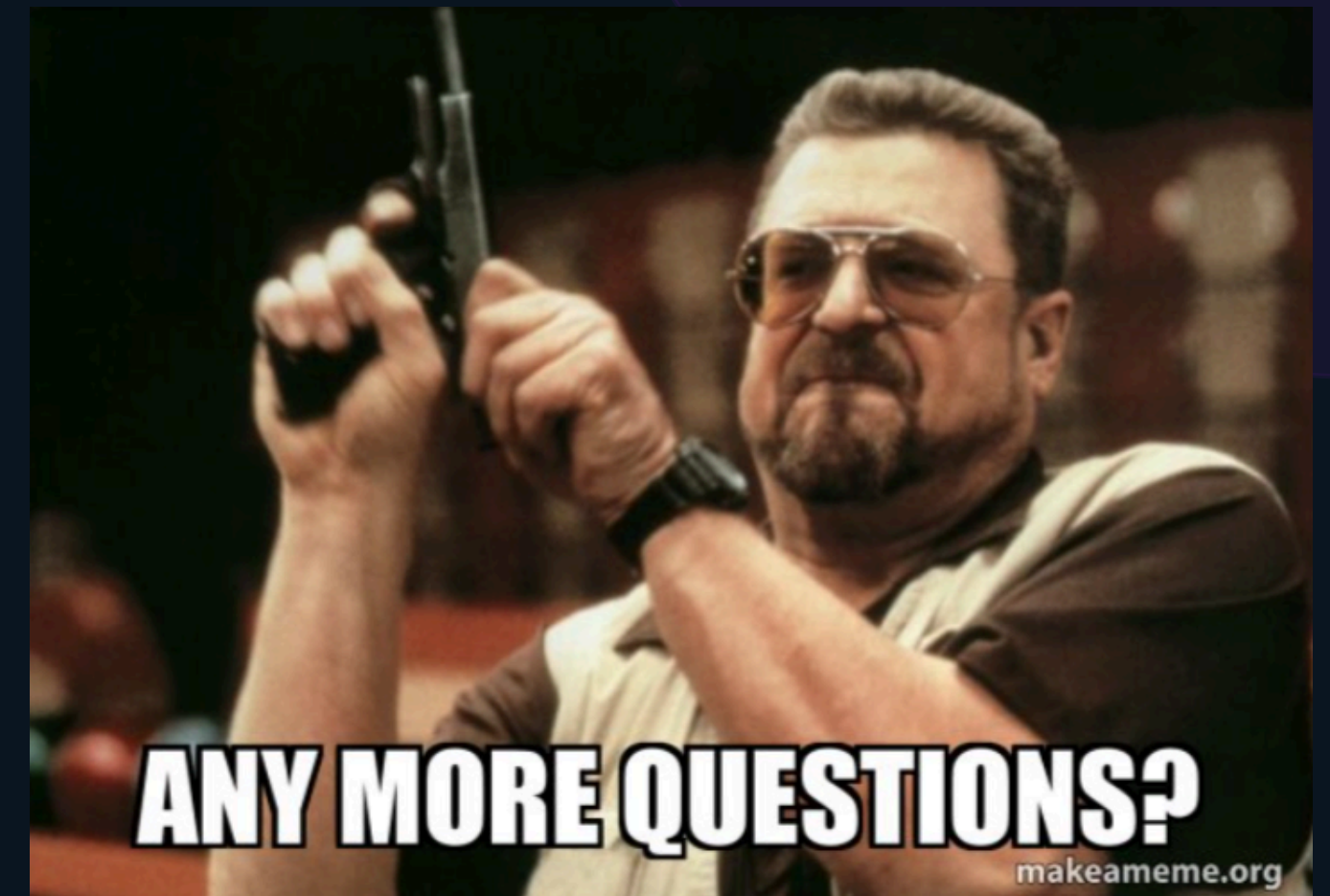
Security

- Self-Healing DB Ownership
- Login Privilege Audit Report
- SQL Server Audit on Employees
- Always Encrypted Salary + TDE

PROJECT COMPLETE

Enterprise Database Architecture & Governance

THANK YOU!



✓ **Dual SQL Server Instances**

configured and operational

✓ **Automated Maintenance**

Weekend midnight — zero contention

✓ **Tiered Backup Strategy**

Full / Differential / T-Log per tier

✓ **Full Security Stack**

Audit, Encryption, Self-Healing, Governance

✓ **Live Replication Pipeline**

30-min sync + daily reports + reverse sync